HARAMAYA UNIVERSITY | DEPARTMENT OF COMPUTER SCIENCE

CoSc3053: 90-DAY JAVA MASTERY CHALLENGE

DAY 3 WORKSHEET: ARRAY UTILITIES & ALGORITHMIC SEARCH (ArrayUtils)

Student Name: _____	**Student ID:** _____
Target Completion Date: Day 3 / 90 Days	**Lab Section / Group:** Section ____ / Group ____
Environment Status: JDK 17/21/25 LTS Verified [X]	**Evaluation Score:** _____ / 10.0 Marks

1. Technical Objectives & Conceptual Core

Concept Dimension	Technical Specification & Array Invariants
Array Traversal & Min/Max Isolation	Linear search O(n) traversing contiguous Heap memory. Requires tracking candidate extrema initialized to array[0].
In-Place Array Reversal	Two-pointer swap mechanism (left=0, right=len-1) operating in O(n) time and O(1) auxiliary space without secondary array allocation.
Binary Search Algorithm	Divide-and-conquer search O(log n) operating strictly on pre-sorted arrays. Calculates mid = low + (high - low) / 2 to prevent integer overflow.

2. Executable Java Source Code (ArrayUtils.java)

```
package edu.haramaya.cosc3053.day3;

import java.util.Arrays;

/**
 * Day 3 Challenge: Array Utilities & Algorithmic Search Engine
 * Demonstrates min/max calculation, in-place reversal, and binary search.
 */
public class ArrayUtils {

    // 1. Find Minimum Element in O(n) time
    public static int findMin(int[] arr) {
        if (arr == null || arr.length == 0) throw new IllegalArgumentException("Array cannot be empty or
null");
        int min = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < min) min = arr[i];
        }
        return min;
    }

    // 2. Find Maximum Element in O(n) time
    public static int findMax(int[] arr) {
        if (arr == null || arr.length == 0) throw new IllegalArgumentException("Array cannot be empty or
null");
        int max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) max = arr[i];
        }
        return max;
    }

    // 3. In-Place Array Reversal O(n) time, O(1) space
    public static void reverse(int[] arr) {
        if (arr == null) return;
        int left = 0, right = arr.length - 1;
        while (left < right) {
            int temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
    }

    // 4. Binary Search O(log n) time (Requires Pre-Sorted Array)
    public static int binarySearch(int[] arr, int target) {
        int low = 0, high = arr.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2; // Prevents overflow
            if (arr[mid] == target) return mid;
            if (arr[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
        return -1; // Target not found
    }

    // Test Runner Method
    public static void executeRun(String runName, int[] inputArr, int searchTarget) {
        System.out.println("========================================");
        System.out.println(" RUN: " + runName);
        System.out.println("========================================");
        System.out.println(" Original Array : " + Arrays.toString(inputArr));
        System.out.println(" Min Element    : " + findMin(inputArr));
        System.out.println(" Max Element    : " + findMax(inputArr));
```

```
        int[] reversed = inputArr.clone();
        reverse(reversed);
        System.out.println(" Reversed Array : " + Arrays.toString(reversed));

        int[] sorted = inputArr.clone();
        Arrays.sort(sorted);
        int idx = binarySearch(sorted, searchTarget);
        System.out.println(" Sorted Array   : " + Arrays.toString(sorted));
        System.out.println(" Binary Search ('" + searchTarget + "'): Index " + idx + (idx != -1 ? "
[FOUND]" : " [NOT FOUND]"));
        System.out.println();
    }

    public static void main(String[] args) {
        // Sample Run 1: Standard Unsorted Array
        executeRun("Sample Run 1 (Standard Elements)", new int[]{45, 12, 89, 33, 67, 21}, 33);

        // Sample Run 2: Negative Numbers & Duplicates
        executeRun("Sample Run 2 (Negatives & Duplicates)", new int[]{-15, 0, 42, -8, 42, -15}, -8);

        // Sample Run 3: Single-Element Array
        executeRun("Sample Run 3 (Single Element)", new int[]{100}, 100);
    }
}
```

3. Visual Execution Snapshots (3 Sample Runs)

```
========================================
 RUN: Sample Run 1 (Standard Elements)
========================================
 Original Array : [45, 12, 89, 33, 67, 21]
 Min Element    : 12
 Max Element    : 89
 Reversed Array : [21, 67, 33, 89, 12, 45]
 Sorted Array   : [12, 21, 33, 45, 67, 89]
 Binary Search ('33'): Index 2 [FOUND]

========================================
 RUN: Sample Run 2 (Negatives & Duplicates)
========================================
 Original Array : [-15, 0, 42, -8, 42, -15]
 Min Element    : -15
 Max Element    : 42
 Reversed Array : [-15, 42, -8, 42, 0, -15]
 Sorted Array   : [-15, -15, -8, 0, 42, 42]
 Binary Search ('-8'): Index 2 [FOUND]

========================================
 RUN: Sample Run 3 (Single Element)
========================================
 Original Array : [100]
 Min Element    : 100
 Max Element    : 100
 Reversed Array : [100]
 Sorted Array   : [100]
 Binary Search ('100'): Index 0 [FOUND]
========================================
```

4. Student Self-Guided Checkpoints & Analysis Questions

Task / Analysis Question	Student Analysis & Code Proof Response
1. Binary Search Pre-Condition: **Why does binary search produce incorrect indices or -1 if executed on an unsorted array?**	_____ _____
2. Two-Pointer In-Place Reversal: **Explain why swap(arr[left], arr[right]) uses O(1) auxiliary memory compared to creating a new result array.**	_____ _____
3. Overflow Guard in Midpoint Calc: **Why is mid = low + (high - low) / 2 preferred over mid = (low + high) / 2 for large arrays?**	_____ _____

5. Evaluation Rubric & Sign-Off Sheet

Evaluation Dimension	Max Marks	Awarded Marks & Remarks
Min / Max Traversal Logic & Guard Checks	2.5 Marks	_____ / 2.5 Marks
In-Place Two-Pointer Reversal Correctness	2.5 Marks	_____ / 2.5 Marks
Binary Search Implementation & Overflow Prevention	2.5 Marks	_____ / 2.5 Marks
Self-Guided Questions & Code Proof Analysis	1.5 Marks	_____ / 1.5 Marks
Lab Defense & Oral Code Walkthrough	1.0 Mark	_____ / 1.0 Mark

Signature: _____ **Date: _____**